ACIT 2515 Test 1: Reference Sheet

This sheet is a compact reference. The railroad diagrams show common course
patterns, not the full Python grammar.

Variables

Purpose

Use variables to give names to values so they can be read, updated, and passed
between parts of a program.

Notes

- Numeric: Used for arithmetic and measured quantities.
  - `int` : whole numbers, such as counts and port numbers.
  - `float` : decimal numbers, such as percentages and measurements.
  - `complex` : numbers with real and imaginary parts.
- Sequence: ordered values that can be iterated over and accessed by position.
  - `str` : text characters.
  - `list` : an ordered collection that can be changed.
  - `tuple` : an ordered collection that should remain fixed.
  - `range` : a generated sequence of integers, often used for repetition.
- Boolean: used for true/false conditions
  - `bool` is `True` or `False` .
- Set: a changeable collection of unique values, used for membership tests.
  - `set` : stores distinct values without positional indexes.
- Mapping: associates keys with values for lookup.
  - `dict` : stores key-value pairs, such as a hostname mapped to its status.
- None type: `None` represents the absence of a value.

Examples

```
# These declarations follow assignment; annotations document intended types.
hostname: str = "server01"     # "server01"
port: int = 443                # 443
usage_percent: float = 82.5    # 82.5
is_online: bool = True         # True
owner: str | None = None       # None
```

Operators

Arithmetic Operators

Purpose

Use arithmetic operators to calculate numeric values and measurements.

Notes

- Unary `+x` : leaves a numeric value positive.
- Unary `-x` : negates a numeric value.
- Binary `x + y` : adds two numbers.
- Binary `x - y` : subtracts the right number from the left number.
- `x * y` : multiplies two numbers.
- `x / y` : divides and returns a floating-point result; division by zero raises
  `ZeroDivisionError` .
- `x // y` : performs floor division, rounding down; division by zero raises `ZeroDivisionError` .
- `x % y` : returns the remainder after division; a zero divisor raises `ZeroDivisionError` .
- `x ** y` : raises `x` to the power of `y` .

Examples

```
total_gb = 500.0               # 500.0
used_gb = 125.5                # 125.5
available_gb = total_gb - used_gb  # 374.5
half_used = used_gb / 2        # 62.75
whole_groups = 17 // 5         # 3
remainder = 17 % 5             # 2
square = 4 ** 2                 # 16
negative_offset = -3            # -3
```

Comparison Operators

Purpose

Compare values and produce a Boolean result.

Notes

- `x == y` : tests whether two values are equal.
- `x != y` : tests whether two values are not equal.
- `x < y` : tests whether `x` is less than `y` .
- `x <= y` : tests whether `x` is less than or equal to `y` .
- `x > y` : tests whether `x` is greater than `y` .
- `x >= y` : tests whether `x` is greater than or equal to `y` .
- `=` assigns a value; it is not a comparison operator.
- Comparisons can be chained, as in `1 <= port <= 1024` .

Examples

```
port = 443                     # 443
port == 443                    # True
1 <= port <= 1024              # True
is_different = port != 80      # True
```

Boolean Operators

Purpose

Combine truth values to control decisions and repeated actions.

Notes

- `and` is true when both operands are truthy
- `or` is true when either operand is truthy
- `not` reverses truth and returns a boolean

Examples

```
is_online = True                          # True
is_secure_port = True                     # True
should_check = is_online and is_secure_port   # should_check is True
unavailable = not is_online or port != 443    # unavailable is False
```

Truthy and Falsy Values

Purpose

Understand how Python treats values as true or false in conditions and Boolean
expressions.

Notes

- Python considers an object truthy by default unless its type defines it as falsy. See the
  Python truth value testing documentation.
- Falsy sequences and collections include `[]` , `()` , `{}` , `set()` , `""` , and `range(0)` .
- Falsy numeric values include `0` , `0.0` , and `0j` .
- `None` and `False` are falsy.
- Non-empty sequences and collections, non-zero numbers, and `True` are truthy.
- Use `bool(value)` to inspect the truth value explicitly.

Examples

```
# Falsy values do not run the indented body of an if statement.
a = 0
if a:
  print(a)  # No output because 0 is falsy.

# Non-zero numbers are truthy, so the body runs.
a = 5
if a:
  print(a)  # 5

empty_services = []
zero_value = 0
no_owner = None

bool(empty_services)  # False
bool(zero_value)      # False
bool(no_owner)        # False
bool(["DNS"])         # True
bool(5)               # True
```

Membership Operators

Purpose

Test whether a value occurs in a sequence or collection.

Notes

- For a dictionary, membership tests check keys.
- `in` and `not in` produce Boolean results.

Examples

```
open_ports = [22, 443]
needs_https_check = 443 in open_ports       # True
port_is_available = 8080 not in open_ports  # True
```

Identity Operators

Purpose

Test whether two names refer to the same object.

Notes

- Use `is` and `is not` for object identity, not value equality.
- Use `value is None` to test for `None`; use `==` for value equality.

Examples

```
x = [ 1, 2, 3]
y = [ 1, 2, 3]
z = x
x == y                          # True
x is y                          # False
x == z                          # True
x is z                          # True
x.append(4)
x                               # [1,2,3,4]
y                               # [1,2,3]
z                               # [1,2,3,4]
owner = None                    # None
owner is None                   # True
```

Conditional Expressions

Purpose

Choose one value from two alternatives based on a condition.

Notes

- Use a conditional expression for a simple choice.
- Use a regular `if` statement for multiple steps or branches.

Syntax

railroad-ebnf-beta title "Conditional Expression" conditional = value_if_true "if" condition "else" value_if_false ;

Examples

```
status = "online" if is_online else "offline"  # "online"
```

Sequence Operators

Purpose

Concatenate or repeat strings, lists, and tuples.

Notes

- `+` concatenates two sequences.
- `*` repeats a sequence by an integer.
- These operations create a new sequence.

Syntax

railroad-ebnf-beta title "Sequence Operators" sequence_operation = sequence "+" sequence | sequence "*" integer | integer "*" sequence ;

Examples

```
label = "host: " + hostname       # "host: server01"
separator = "-" * 8                # "--------"
services = ["DNS"] + ["SSH"]       # ["DNS", "SSH"]
```

Augmented Assignment Operators

Purpose

Update a variable using an arithmetic operation and assign the result back.
TODO: add content for all operators in non assigned operators

Examples

```
attempt = 1       # 1
attempt += 1      # 2
attempt *= 2      # 4
```

Notes

- `attempt += 1` is equivalent to `attempt = attempt + 1`.
- The available augmented assignments correspond to the arithmetic operators
  covered above.

Operator Precedence

Purpose

Determine which operators are evaluated first when parentheses are absent.

Order

The following order is highest precedence to lowest precedence:

1. Parenthesized expressions and displays: `( ... )`, `[ ... ]`, `{key: value}`, `{value, ...}`
2. Subscription, slicing, calls, and attribute references: `x[index]`, `x[start:stop]`, `x(arguments)`, `x.attribute`
3. Await expression: `await x`
4. Exponentiation: `**`
5. Unary positive, negative, and bitwise NOT: `+x`, `-x`, `~x`
6. Multiplication, matrix multiplication, division, floor division, and remainder: `*`, `@`, `/`, `//`, `%`
7. Addition and subtraction: `+`, `-`
8. Shifts: `<<`, `>>`
9. Bitwise AND: `&`
10. Bitwise XOR: `^`
11. Bitwise OR: `|`
12. Comparisons, membership, and identity: `<`, `<=`, `>`, `>=`, `!=`, `==`, `in`, `not in`, `is`, `is not`
13. Boolean `not`
14. Boolean `and`
15. Boolean `or`
16. Conditional expression: `value_if_true if condition else value_if_false`
17. Lambda expression: `lambda parameters: expression`
18. Assignment expression: `name := expression`

Notes

- Operators at the same level generally evaluate left to right, except exponentiation and conditional expressions, which group right to left.
- Comparisons, membership tests, and identity tests have the same precedence and
  can be chained.
- Parentheses make grouping explicit.

Interactive CLI Input

Purpose

Get data interactively via the command line.

Notes

- `input()` always returns a `str`, even when the user types digits.
- Convert numeric text with `int()` or `float()`.
- Handle invalid text with `ValueError` when the program can recover.

Syntax

Diagram

railroad-ebnf-beta title "Accepting Input" input_call = "input" "(" ["prompt"] ")" ;

Examples

```
# input reads the host and port as strings.
# Convert the port in try because invalid numeric text raises ValueError.
hostname = input("Host name: ")
port_text = input("Port number: ")

try:
  port = int(port_text)
except ValueError:
  print("Enter the port as a whole number.")
else:
  print(f"Checking {hostname} on port {port}")  # Example: Checking server01 on p
```

Conditional

Purpose

Use conditional statements to choose which block of code runs based on a condition.

Notes

- Indentation defines each block.
- A colon begins the indented block.

Syntax

Diagram

railroad-ebnf-beta title "Conditional Statements" if_statement = "if" "condition" ":" "block" ("elif" "condition" ":" "block")* ["else" ":" "block"] ;

Examples

```
if usage_percent >= 90:
  # Statements in the if block must be indented to the same level
  status = "critical"
elif usage_percent >= 75: # Follows if_statement: elif may repeat
  status = "warning"
else:                     #the final else is optional.
  status = "normal"
# With usage_percent = 82.5, status is "warning".
```

for Loops

Purpose

Use a `for` loop to process every item in a collection or sequence.

Syntax

Diagram

railroad-ebnf-beta title "for Loop Syntax" for_statement = "for" target "in" iterable ":" "block" ["else" ":" "block"] ; target = "variable (or pattern) that receives each value from the iterable during iteration" ; iterable = "collection" | range_call | enumerate_call | zip_call ; range_call = "range" "(" ("stop" | "start" "," "stop") ["," "step"] ")" ; enumerate_call = "enumerate" "(" "iterable" ["," "start" "=" "integer"] ")" ; zip_call = "zip" "(" "iterable" ("," "iterable")+ ")" ; loop_control = "break" | "continue" ;

Examples

```
for char in "test":
  print(char) #iterates over string printing one char per line

for attempt in range(1, 4):
  print(f"Connection attempt {attempt}")  # Attempts 1, 2, and 3

services = ["Spooler", "W32Time", "WinRM"]
for service_name in services:
  print(f"Checking {service_name}")  # Checking each service name

# enumerate() supplies an index and item during each iteration.
for position, service_name in enumerate(services, start=1):
  print(f"{position}. {service_name}")  # 1. Spooler, 2. W32Time, 3. WinRM

# zip() pairs items from multiple iterables in order.
statuses = ["running", "stopped", "running"]
for service_name, service_status in zip(services, statuses):
  print(f"{service_name}: {service_status}")

# Outputs
# Spooler: running
# W32time: stopped
# WinRM: running

for service_name in ["DNS", "SSH", "NTP", "HTTPS", "SMTP"]:
  if service_name == "SSH":
    continue # skips the rest of the current for loop.
  if service_name == "HTTPS":
    break # stops the nearest loop immediately.
  print(service_name)

# Outputs:
# DNS
# NTP
```

Notes

- the `else` block is only executed if a for loop completes without an exception
  or `break`
- `break` exits the nearest enclosing loop immediately; the loop does not process any remaining items.
- `continue` skips the rest of the current iteration and starts the next one.
- The stop value in `range()` is exclusive
- `enumerate(iterable, start=0)` returns an iterator of `(index, item)` pairs.
  The default index is `0`; use `start=1` for human-readable numbering.
- `zip(iterable1, iterable2, ...)` returns an iterator of tuples containing one
  item from each input iterable at the same position.
- `zip()` stops when the shortest input is exhausted; extra items in longer inputs are not included.
- Both `zip` and `enumerate` return iterators, and can be used in a `for` loop
  or converted to a list with `list()`.

while Loops

Purpose

Use a `while` loop to repeat a block while a condition remains true, especially
when the number of repetitions is not known in advance.

Syntax

Diagram

railroad-ebnf-beta title "while Loop Syntax" while_statement = "while" "condition" ":" "block" ["else" ":" "block"] ;

Examples

```
attempt = 1
while attempt <= 3:
  print(attempt)  # 1, then 2, then 3
  # incrementing attempt lets the condition become false, otherwise it will iterate forever
  attempt += 1
else:
  print("Out of Attempts") #executes after the while loop exists normally
```

Notes

- Update a value used by the condition so it can eventually become false; otherwise, the loop may never stop.

Sequences

Common Sequence Features

Strings, lists, tuples, and `range` objects are ordered sequences. They support
iteration, `len(sequence)`, membership tests with `value in sequence`, and
indexing from zero. Slices select a range of positions; the stop index is excluded. Strings, tuples, and `range` objects are immutable, while lists can be
changed.

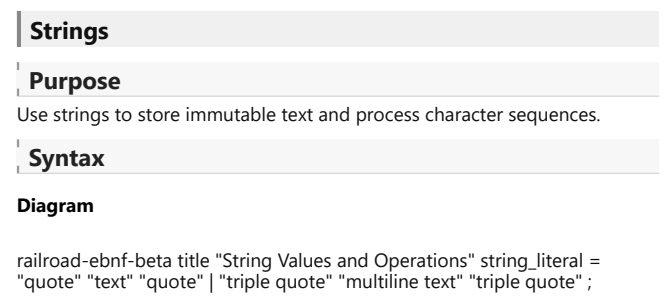
Strings

Purpose

Use strings to store immutable text and process character sequences.

Syntax

Diagram



railroad-ebnf-beta title "String Values and Operations" string_literal = "quote" "text" "quote" | "triple quote" "multiline text" "triple quote" ; string_operation = string "+" string | string "*" integer | value ("in" | "not in") string ; string_access = string "[" index "]" | string "[" start ":" stop [":" step] "]" ; string_conversion = "len" "(" string ")" | "str" "(" value ")" | "repr" "(" value ")" ;

Examples

```
# Strings are immutable sequences; operations return new values.
single_line = "server01"
multi_line = """line one
line two"""
escaped = "line one\nline two"       # Contains a newline.
raw_path = r"C:\\logs\\startup.log"  # Backslashes remain literal.

label = "host: " + single_line       # "host: server01"
separator = "-" * 8                  # "--------"
has_server = "server" in single_line  # True
has_router = "router" not in single_line  # True
length = len(single_line)            # 8
first = single_line[0]               # "s"
last = single_line[-1]               # "1"
prefix = single_line[:4]             # "serv"
suffix = single_line[4:]             # "er01"
every_second = single_line[::2]      # "svr0"
as_text = str(443)                   # "443"
debug_text = repr("server01")        # 'server01'
```

Notes

- Strings are immutable sequences; indexing starts at zero and negative indexes
  count from the end.
- Slices use `[start:stop:step]`; the stop index is excluded.
- `+` concatenates strings and `*` repeats a string by an integer.
- `in` and `not in` test for substring membership.
- `len()` counts characters; `str()` converts a value to user-facing text and
  `repr()` returns a developer-oriented representation.
- Escape sequences such as `\n` and `\t` have special meanings; raw strings
  such as `r"C:\\logs\\startup.log"` keep backslashes literal.

F-Strings

Purpose

Use f-strings to build readable text that includes variable values and formatted
numbers.

Syntax

Diagram

railroad-ebnf-beta title "f-String and Format Specification" f_string = "f" quote text_part* quote ; text_part = "literal text" | replacement_field ; replacement_field = "{" expression [":" format_spec] "}" ; format_spec = [[fill] alignment] [width] ["." precision] [type] ; fill = "fill character" ; alignment = "#lt;" | ">" | "^" ; width = "minimum number of characters" ; precision = "number of digits after decimal point" ; type = "f" | "d" | "s" ;

Examples

```
# An f-string starts with f; braces insert values; : begins an optional format.
hostname = "server01"
disk_usage = 82.567

print(f"Checking {hostname}")       # Insert a value
print(f"Usage: {disk_usage:.2f}%")   # Two digits after the decimal: 82.57%
print(f"{disk_usage:08.2f}")        # Width 8, zero-filled: 0082.57
print(f"|{hostname:<12}|")          # Width 12, left aligned
print(f"|{hostname:>12}|")          # Width 12, right aligned
print(f"|{hostname:^12}|")          # Width 12, centered
# Output includes Checking server01, Usage: 82.57%, and aligned fields.
```

Notes

- A format width is a minimum field width in characters; longer values are not truncated.
- Alignment options are `<` for left, `>` for right, and `^` for centered.
- For floating-point values, `.2f` displays two digits after the decimal point.
  The `f` presentation type selects fixed-point formatting.

Lists

Purpose

Use a list for an ordered group of values that may change and may contain duplicates.

Notes

- Lists are ordered and mutable
- Sequence indexes start at zero, and slices exclude their `stop` position.
- `.append(value)` adds one object to the end and returns `None`.
- `.clear()` removes all items and returns `None`.
- `.copy()` returns a shallow copy of the list.
- `.count(value)` returns the number of occurrences of a value.
- `.extend(iterable)` appends every item from another iterable and returns `None`.
- `.index(value, start=0, stop=...)` returns the first matching index and raises `ValueError` when the value is absent.
- `.insert(index, value)` inserts before the requested index and returns `None`.
- `.pop(index=-1)` removes and returns an item; it raises `IndexError` for an empty list or invalid index.
- `.remove(value)` removes the first matching item and raises `ValueError` when the value is absent.
- `.reverse()` reverses the list in place and returns `None`.
- `.sort(key=None, reverse=False)` sorts the list in place, returns `None`, and is stable. A `key` function controls comparison values; `reverse=True` sorts in descending order.
- Use `len(items)` to count items and `value in items` to check membership.

Syntax

Diagram

railroad-ebnf-beta title "List Syntax" list_literal = "[" [value ("," value)* [","]] "]" ; list_access = "list name" "[" index "]" | "list name" "[" start ":" stop [":" step] "]" ;

Examples

```
services = ["DNS", "SSH", "HTTPS"]       # Initial list
first_service = services[0]              # "DNS"
last_service = services[-1]              # "HTTPS"
first_two = services[0:2]                # ["DNS", "SSH"]
services.append("NTP")                   # Adds NTP at the end
services[1] = "SFTP"                     # Updates the second item
services.insert(1, "NTP")                # Inserts NTP at index 1
removed_service = services.pop()         # Removes and returns the last item

service_count = services.count("NTP")    # Number of NTP entries: 1
service_index = services.index("HTTPS")  # First index of HTTPS: 2
services.extend(["HTTP", "FTP"])         # Adds items from another list
service_copy = services.copy()           # Shallow copy
services.remove("FTP")                   # Removes the first FTP entry
services.reverse()                       # Reverses the list in place
services.sort()                          # Sorts the list in place
services.clear()                         # Removes all items; []
```

Tuples

Purpose

Use a tuple to group related values that should be treated as a fixed sequence.

Notes

- Tuples are ordered and immutable; their items cannot be added, removed, or replaced.
- Indexing and slicing work like lists.
- Tuples support iteration, `len()`, membership testing, `.count(value)`, and `.index(value)`.

Syntax

Diagram

railroad-ebnf-beta title "Tuple Syntax" tuple_literal = "(" ")" | "(" value "," [value ("," value)* [","]] ")" ; tuple_access = "tuple name" "[" index "]" | "tuple name" "[" start ":" stop [":" step] "]" ;

Examples

```
# Demonstrates tuple_literal and tuple_access; (443,) needs its trailing comma.
location = ("server01", "Vancouver")  # Tuple of two strings
single_port = (443,)                  # One-item tuple
server_name = location[0]             # "server01"
host_name, site_name = location       # "server01", "Vancouver"
```

Ranges

Purpose

Use `range` to represent a sequence of integers for repetition, counting, and index-based iteration without writing out each integer.

Syntax

Diagram

railroad-ebnf-beta title "range Call" range_call = "range" "(" (stop | start "," stop) ["," step] ")" ;

Examples

```
# Follows range_call; stop is excluded, so range(1, 4) yields 1, 2, 3.
for attempt in range(1, 4):
  print(attempt)  # 1, 2, 3
```

Notes

- Common forms are `range(stop)`, `range(start, stop)`, and `range(start, stop, step)`.
- A `range` supports iteration, `len()`, indexing, and membership tests.
- A `range` is immutable; its values cannot be changed.

Sets

Purpose

Use a set to keep unique values or test membership without relying on position.

Syntax

Diagram

railroad-ebnf-beta title "Set Syntax" set_literal = "set" "(" ")" | "{" value ("," value)* [","] "}" ; set_update = "set name" "." ("add" | "clear" | "discard" | "difference_update" | "intersection_update" | "pop" | "remove" | "symmetric_difference_update" | "update") "(" [value] ")" ; set_query = "set name" "." ("isdisjoint" | "issubset" | "issuperset") "(" "other set" ")" ; set_copy = "set name" ".copy" "(" ")" ; set_algebra = "set name" "." ("difference" | "intersection" | "symmetric_difference" | "union") "(" "other set" ")" ;

Examples

```
# Demonstrates set_literal and set_update; duplicate 443 is stored once.
unique_ports = {22, 443, 443}
unique_ports.add(80)              # {22, 80, 443}
has_https = 443 in unique_ports  # True
empty_set = set()                 # set()

web_ports = {80, 443}
secure_ports = {443, 8443}
shared_ports = web_ports.intersection(secure_ports)
# shared_ports is {443}.
```

Notes

- Sets are mutable and have no positional indexes.
- Use `set()` to create an empty set; `{}` creates an empty dictionary.
- `.add(value)`: adds one value to the set.
- `.clear()`: removes every value from the set.

- `.copy()` : returns a shallow copy of the set.
- `.difference(other)` : returns values in this set but not in `other` .
- `.difference_update(other)` : removes values found in `other` in place.
- `.discard(value)` : removes a value if present; otherwise does nothing.
- `.intersection(other)` : returns values present in both sets.
- `.intersection_update(other)` : keeps only shared values in place.
- `.isdisjoint(other)` : returns `True` when the sets share no values.
- `.issubset(other)` : returns `True` when every value is in `other` .
- `.issuperset(other)` : returns `True` when this set contains every value in
  `other` .
- `.pop()` : removes and returns an arbitrary value; it raises `KeyError` for an
  empty set.
- `.remove(value)` : removes a value and raises `KeyError` if it is absent.
- `.symmetric_difference(other)` : returns values in exactly one of the sets.
- `.symmetric_difference_update(other)` : keeps only values in exactly one set,
  modifying this set in place.
- `.union(other)` : returns a new set containing values from both sets.
- `.update(iterable)` : adds values from an iterable to the set.

Dictionaries

Purpose

Use a dictionary to associate values with unique, meaningful keys for lookup.

Notes

- Dictionary keys must be hashable; values can be duplicated.
- Access entries by key, not numeric position.
- `.clear()` : removes all key-value pairs and returns `None` .
- `.copy()` : returns a shallow copy of the dictionary.
- `.fromkeys(iterable, value=None)` : creates a new dictionary using the iterable's values as keys and the same default value for each key.
- `.get(key, default=None)` : returns a value or the default without raising
  `KeyError` when the key is absent.
- `.items()` : returns a view of `(key, value)` pairs.
- `.keys()` : returns a view of the dictionary's keys.
- `.pop(key)` : removes and returns the value for a key; it raises `KeyError` if
  the key is absent unless a default is supplied.
- `.popitem()` : removes and returns the most recently inserted `(key, value)`
  pair; it raises `KeyError` when the dictionary is empty.
- `.setdefault(key, default=None)` : returns the value for a key, inserting the
  default if the key is absent.
- `.update(mapping_or_iterable)` : adds or replaces key-value pairs in place and
  returns `None` .
- `.values()` : returns a view of the dictionary's values.
- Iterating over a dictionary directly is equivalent to iterating over its keys.
- `.keys()` supplies keys, `.values()` supplies values, and `.items()` supplies
  `(key, value)` pairs for unpacking in a `for` loop.
- Use `len(mapping)` to count entries.

Syntax

Diagram

railroad-ebnf-beta title "Dictionary Syntax" dict_literal = "{" [key ":" value ("," key ":" value)* [","]] "}" ; dict_access = "dict name" "[" key "]" | "dict name" ".get" "(" key ["," default] ")" ; dict_update = "dict name" "[" key "]" "=" value | "dict name" ".update" "(" "mapping" ")" ; dict_iteration = "for" "key" "in" "dict name" ".keys" "(" ")" ":" "block" | "for" "value" "in" "dict name" ".values" "(" ")" ":" "block" | "for" "key" "," "value" "in" "dict name" ".items" "(" ")" ":" "block" ;

Examples

```
host = {"hostname": "server01", "ip": "192.168.1.10"}
host["port"] = 443                    # Adds the port field.
hostname = host["hostname"]           # "server01"
owner = host.get("owner", "unassigned")  # "unassigned"
host["status"] = "online"             # Adds the status field.

for field_name, field_value in host.items():
  print(f"{field_name}: {field_value}")  # Prints each field and value.

for field_name in host.keys():
  print(field_name)  # Prints each key.

for field_value in host.values():
  print(field_value)  # Prints each value.
```

Functions

Purpose

Use functions to package reusable behavior behind named inputs and return
values.

Notes

- Parameters are names in a function definition; arguments are values in a function call.
- `return` sends a result back. Without `return` , a function returns `None` .
- A function that receives a list can mutate the shared list; those changes are
  visible to the caller.
- Reassigning the parameter name only changes the local name. Use `items.copy()`
  before changing a private copy.

Syntax

Diagram

railroad-ebnf-beta title "Function Definition and Call" function_definition = "def" "function name" "(" [parameter ("," parameter)* [","]] ")" ["->" "return type"] ":" function_block ; parameter = "name" [":" "type"] ["=" "default value"] ; function_block = "one or more indented statements" ; function_call = "function name" "(" [argument ("," argument)* [","]] ")" ; argument = "value" | "name" "=" "value" ;

Examples

```
def free_space(total_gb: float, used_gb: float) -> float:
  """Return the available disk space in gigabytes."""
  return total_gb - used_gb


available = free_space(total_gb=500.0, used_gb=125.5)  # 374.5

def host_label(hostname: str, site: str | None = None) -> str:
  """Return a hostname label, optionally including its site."""
  if site is None:
    return hostname  # Example: "server01"
  return f"{hostname} ({site})"  # Example: "server01 (Vancouver)"

# Passing a list shares the mutable object, so appending affects the caller's lis
def add_service(services: list[str], service_name: str) -> None:
  """Add a service name to the caller's list."""
  services.append(service_name)

# Alterantively make a copy, so the caller's list is unaffected
def sorted_service_copy(services: list[str]) -> list[str]:
  """Return a sorted copy without changing the caller's list."""
  result = services.copy()
  result.sort()
  return result

# The copy is sorted separately, leaving service_names in its original order.
service_names = ["WinRM", "DNS"]
add_service(service_names, "SSH")
sorted_names = sorted_service_copy(service_names)  # ["DNS", "SSH", "WinRM"]
# service_names remains ["WinRM", "DNS", "SSH"].
```

Type Annotations and Docstrings

Purpose

Use type annotations to describe expected values and docstrings to document a
function's purpose, parameters, return value, and possible exceptions.

Notes

- Parameter annotations describe expected argument values; the return annotation describes the function result.
- Common collection annotations include `list[str]` , `dict[str, int]` , `tuple[str, int]` , and `set[str]` .
- `str | None` means a string or `None` ; a default such as `site = None` makes
  the argument optional for callers.
- Google-style docstrings begin with a summary and may include `Args:` , `Returns:` , and `Raises:` sections.
- Annotations guide readers and type checkers; Python does not automatically
  enforce them at runtime.

Syntax

Diagram

railroad-ebnf-beta title "Function Contract" annotated_parameter = "name" ":" "type" ["=" "default value"] ; return_annotation = "->" "return type" ; docstring = "triple-quoted summary and sections" ; function_contract = "def" "name" "(" annotated_parameter* ")" [return_annotation] ":" docstring "body" ;

Examples

```
def host_label(hostname: str, site: str | None = None) -> str:
  """Return a hostname label, optionally including its site.

  Args:
    hostname: Host name to label.
    site: Optional site name.

  Returns:
    The hostname, optionally followed by the site.
  """
  if site is None:
    return hostname  # "server01"
  return f"{hostname} ({site})"  # "server01 (Vancouver)"
```

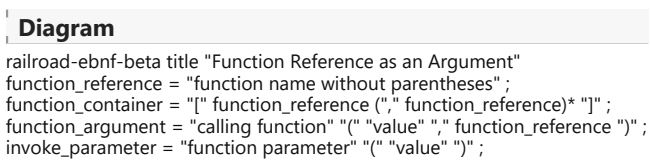
Functions as Values and Arguments

Purpose

It is possible to use a function as a value when behavior should be selected or passed into
another function for that function to call.

Syntax

Diagram



Examples

```
def to_upper(text):
    return text.upper()

def to_lower(text):
    return text.lower()

def add_exclamation(text):
    return text + "!"

# Function to apply a transformation
def greet(text, transform_func):
    # Apply the passed-in text-changing function
    transformed_text = transform_func(text)
    return f"Hello, {transformed_text}"

# Examples
print(greet("world", to_upper))        # Output: Hello, WORLD
print(greet("WORLD", to_lower))        # Output: Hello, world
print(greet("friend", add_exclamation))# Output: Hello, friend!
```

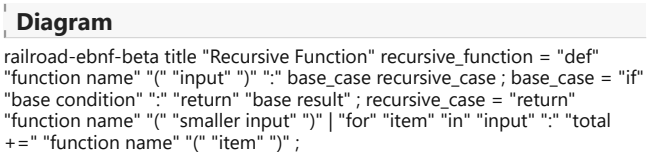
Recursion

Purpose

Use recursion to solve a problem by applying the same function to smaller or
simpler parts of the input.

Syntax

Diagram



Examples

```
def sum_nested(data: int | list) -> int:
  """Return the sum of integers in a possibly nested list."""
  # Base case: integers return directly; the loop recursively sums child items.
  if isinstance(data, int):  # Base case
    return data

  total = 0
  for item in data:  # Recursive case: process each child
    total += sum_nested(item)
  return total  # Example: sum_nested([1, [2, 3]]) returns 6.
```

Notes

- Every recursive function needs a base case that stops and a recursive case
  that moves toward it.
- Each call has its own local variables.
- Missing a base case, or failing to make progress toward it, can cause `RecursionError`.

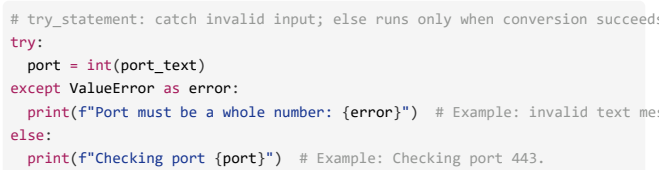
Exceptions

Purpose

Use exceptions to report or handle runtime problems that prevent normal operation, such as invalid input or a failed file operation.

Syntax

Diagram



Examples

```
# try_statement: catch invalid input; else runs only when conversion succeeds.
try:
  port = int(port_text)
except ValueError as error:
  print(f"Port must be a whole number: {error}")  # Example: invalid text message
else:
  print(f"Checking port {port}")  # Example: Checking port 443.
```

```
# raise_statement: validate the value, then let the caller handle ValueError.
def validate_port(port: int) -> int:
  """Return port if it is in the valid range; otherwise raise ValueError."""
  if not 1 <= port <= 65535:
    raise ValueError("port must be between 1 and 65535")
  return port


try:
  checked_port = validate_port(70000)
except ValueError as error:
  print(f"Invalid port: {error}")  # Invalid port: port must be between 1 and 655
```

Notes

- Catch specific exceptions only where the program can recover, and keep `try`
  blocks small.
- `else` runs only if no exception occurs; `finally` runs whether or not one
  occurs.
- Raise an exception when a function cannot fulfill its contract, and let the caller decide how to respond.
- Common exceptions include:
  - `ValueError`: invalid value or conversion.
  - `TypeError`: incompatible types.
  - `IndexError`: invalid sequence position.
  - `KeyError`: missing dictionary key.
  - `FileNotFoundError`: requested file is missing.
  - `PermissionError`: operation is not permitted.
  - `OSError`: operating-system or file operation failure.

pathlib Module

Purpose

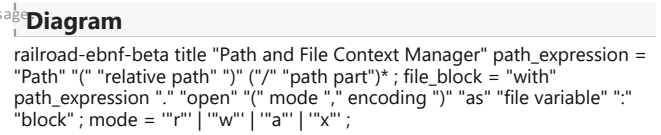
Use `pathlib.Path` to construct portable paths and file context managers to read
or write data while ensuring the file is closed.

Notes

- File modes: `"r"` reads, `"w"` creates or replaces, `"a"` appends, and `"x"`
  creates only if absent.
- Specify UTF-8 for text files.
- Use `.read()` for a small whole file; iterate over a file for line-by-line processing.
- `.write()` does not add a newline automatically.
- Useful `Path` attributes and methods include `.exists()`, `.is_file()`, `.is_dir()`, `.name`, `.suffix`, and `.parent`.

Syntax

Diagram



Examples

```
# Demonstrates path_expression, directory_setup, and file_block.
from pathlib import Path

log_path = Path("logs") / "startup.log"
log_path.parent.mkdir(exist_ok=True)

with log_path.open("a", encoding="utf-8") as log_file:
  log_file.write("System check started\n")

with log_path.open("r", encoding="utf-8") as log_file:
  for line in log_file:
    print(line.strip())  # System check started
```

json Module

Purpose

Use JSON to store and exchange structured data, including nested Python lists
and dictionaries.

Notes

- `json.load(file)` reads JSON from an open file and returns Python data.
- `json.loads(string)` reads JSON from a string and returns Python data.
- `json.dump(data, file)` writes Python data to an open file and returns `None`.
- `json.dumps(data)` converts Python data to a JSON string and returns that
  string.
- Recommended encoding is `encoding="utf-8"` for text files.
- `indent=2` makes written JSON easier to read without taking up too much space.

Syntax

Examples

```
# json_read loads file data into Python; json_write dumps Python data to a file.
import json

with open("tasks.json", "r", encoding="utf-8") as json_file:
  tasks = json.load(json_file)  # Python list or dictionary from the JSON file.

with open("tasks_out.json", "w", encoding="utf-8") as json_file:
  json.dump(tasks, json_file, indent=2)  # Writes formatted JSON; no return value.

json_text = json.dumps(tasks, indent=2)  # JSON text string.
tasks_copy = json.loads(json_text)       # Python list or dictionary.
```

csv Module

Purpose

Use CSV for tabular data stored as rows and columns, especially when data needs
to be opened in spreadsheet software.

Syntax

Diagram

railroad-ebnf-beta title "CSV File Patterns" csv_read = "with open" "(" "path" "," read_mode "," "newline" "=" "empty string" "," encoding ")" "as" "file" ":" reader ; reader = "csv.reader" "(" "file" [reader_options] ")" | "csv.DictReader" "(" "file" ["fieldnames"] ")" ; reader_options = "delimiter" | "quotechar" | "quoting" | "escapechar" ; csv_write = "with open" "(" "path" "," write_mode "," "newline" "=" "empty string" "," encoding ")" "as" "file" ":" writer ; writer = ("csv.writer" "(" "file" [writer_options] ")" | "csv.DictWriter" "(" "file" "," "fieldnames" ")") writer_output ; writer_options = "delimiter" | "quotechar" | "quoting" | "escapechar" ; writer_output = ["writeheader" "(" ")"] ("writerow" | "writerows") ; read_mode = '"r"' ; write_mode = '"w"' ; encoding = "encoding" "=" '"utf-8"' ;

Examples

```
# csv_read uses header names as keys; csv_write takes field names and writes a header.
import csv

with open("hosts.csv", "r", newline="", encoding="utf-8") as csv_file:
  reader = csv.DictReader(csv_file)
  for row in reader:
    print(row["hostname"], row["ip_address"])  # Prints each host and address.

fieldnames = ["hostname", "ip_address"]
with open("report.csv", "w", newline="", encoding="utf-8") as csv_file:
  writer = csv.DictWriter(csv_file, fieldnames=fieldnames)
  writer.writeheader()
  writer.writerow({"hostname": "server01", "ip_address": "192.168.1.10"})  # Writes one row.
```

```
# csv.reader returns each row as a list of strings.
with open("hosts.csv", "r", newline="", encoding="utf-8") as csv_file:
  reader = csv.reader(csv_file)
  for row in reader:
    print(row[0], row[1])  # Prints the hostname and IP address by position.
```

```
# DictReader can use supplied fieldnames when the file has no header row.
fieldnames = ["hostname", "ip_address"]
with open("hosts_without_header.csv", "r", newline="", encoding="utf-8") as csv_file:
  reader = csv.DictReader(csv_file, fieldnames=fieldnames)
  for row in reader:
    print(row["hostname"], row["ip_address"])  # Uses the supplied keys.
```

```
# Configure delimiters and quoting when fields contain separators.
with open("service_report.csv", "w", newline="", encoding="utf-8") as csv_file:
  writer = csv.writer(
    csv_file,
    delimiter=";",
    quotechar='"',
    quoting=csv.QUOTE_MINIMAL,
  )
  writer.writerow(["service", "owner", "status"])
  writer.writerow(["Web; API", "Operations", "running"])
  # Writes semicolon-separated fields and quotes the service field.
```

Notes

- CSV is plain text arranged as rows and fields; the delimiter separates fields
  and the first row commonly contains column names.
- Use `newline=""` when opening CSV files.
- `csv.reader` returns each row as a list; `csv.DictReader` returns each row as
  a dictionary using the header row as keys by default.
- Pass `fieldnames` to `DictReader` when the input has no header row.
- `csv.writer` writes list rows; `csv.DictWriter` writes dictionary rows and
  requires `fieldnames`.
- `DictWriter.writeheader()` writes the field-name row before data rows.
- `delimiter` selects the field separator; common alternatives include comma,
  tab, colon, and semicolon.
- `quotechar` surrounds fields containing delimiters; `quoting` controls when
  fields are quoted, such as `csv.QUOTE_MINIMAL`, `QUOTE_ALL`,
  `QUOTE_NONNUMERIC`, or `QUOTE_NONE`.
- `escapechar` escapes delimiters when quoting is not being used.
- Keep dictionary keys consistent with the writer's `fieldnames`.

`sys.argv` Module

Purpose

Use `sys.argv` to read command-line words supplied when a script is started,
allowing each run to use different input without editing the program.

Notes

- `sys.argv[0]` is the script name; `sys.argv[1:]` contains user-supplied arguments.
- Check the argument list length before indexing it.
- Convert text arguments explicitly when they represent numbers.
- For multiple optional flags, inspect each remaining string and handle supported values with `if` / `elif` branches.
- Use `sys.exit(code)` with `0` for success and a nonzero value when argument
  processing cannot continue.

Syntax

Examples

```
# Demonstrates arguments, argument_check, positional_argument, and optional_flag.
# Command-line values are strings until explicitly converted.
import sys

arguments = sys.argv[1:]
if len(arguments) < 1:
  print("Usage: python show_host.py <hostname> [--verbose]")
  sys.exit(1)

hostname = arguments[0]
verbose = False


if len(arguments) > 1:
  flag = arguments[1]
  if flag == "--verbose":
    verbose = True
  else:
    print(f"Unknown option: {flag}")
    sys.exit(2)

print(f"Checking {hostname}")
if verbose:
  print("Verbose output enabled")  # Prints when --verbose was supplied.
```

`string` Module

Purpose

Use the `string` module's punctuation constant when processing text and removing
punctuation from input.

Syntax

Diagram

railroad-ebnf-beta title "string.punctuation" punctuation_constant = "import string" "string.punctuation" ;

Examples

```
# Use string.punctuation to build excluded characters, then process input with a
import string

excluded = set(string.punctuation + " ")
clean_text = ""
for character in "Server, ONLINE!":
  if character not in excluded:
    clean_text += character.lower()
# clean_text is "serveronline".
```

Notes

- `string.punctuation` contains the punctuation characters used by this demo.

platform Module

Purpose

Use `platform` to read the operating-system and machine details shown in the
Week 1 System Info Reporter.

Syntax

Diagram

railroad-ebnf-beta title "platform Information" platform_call = "platform" "." ("system" | "release" | "version" | "machine") "(" ")" ;

Examples

```
# Each call returns a string describing the local operating system or machine.
import platform

os_name = platform.system()
os_release = platform.release()
os_version = platform.version()
machine_type = platform.machine()
# Each variable contains a string describing the local system.
```

Notes

- Returned values depend on the operating system and hardware.

socket Module

Purpose

Use `socket` in the Week 1 reporter to retrieve the local hostname and resolve
it to an IP address.

Syntax

Diagram

railroad-ebnf-beta title "Local Hostname and Address" hostname_call = "socket.gethostname" "(" ")" ; address_call = "socket.gethostbyname" "(" "hostname" ")" ; resolution_error = "except" "socket.gaierror" ":" "handle unresolved hostname" ;

Examples

```
# gethostname returns the local host; gethostbyname may raise socket.gaierror.
import socket

hostname = socket.gethostname()
try:
  ip_address = socket.gethostbyname(hostname)
except socket.gaierror:
  ip_address = "unknown"
# hostname is local text; ip_address is its resolved address or "unknown".
```

Notes

- The Week 2 System Reporter lab also uses `socket.AF_INET` to identify IPv4
  addresses in network-adapter records returned by `psutil`.

getpass Module

Purpose

Use `getpass` in the Week 1 reporter to get the username of the account running
the program.

Syntax

Diagram

railroad-ebnf-beta title "Current Username" username_call = "import getpass as alias" "alias.getuser" "(" ")" ;

Examples

```
# The Week 1 reporter imports getpass as gp and calls gp.getuser().
import getpass as gp

username = gp.getuser()  # Current account name as text.
```

Notes

- `getuser()` returns the current account name as text.

psutil Package

Purpose

Use the third-party `psutil` package in the Week 2 System Reporter lab to read
selected information about users, network adapters, CPUs, and memory.

Syntax

Diagram

railroad-ebnf-beta title "System Information with psutil" psutil_call = "psutil" "." ("users" | "net_if_addrs" | "cpu_count" | "virtual_memory") "(" ")" ; memory_percent = "psutil.virtual_memory" "(" ")" ".percent" ;

Examples

```
# Lab calls; this loop gathers usernames without a comprehension.
import psutil

logged_in_users = []
for session in psutil.users():
  logged_in_users.append(session.name)

network_adapters = psutil.net_if_addrs()
logical_cpu_count = psutil.cpu_count()
memory_usage_percent = psutil.virtual_memory().percent
# Values contain adapter records, CPU count, and memory usage percentage.
```

Notes

- `psutil` is not part of the standard library; add it to a project with `uv add psutil`.
- The lab uses `socket.AF_INET` with `net_if_addrs()` to select IPv4 adapter
  addresses.

Modules, Imports, and Script Structure

Purpose

Use modules to organize reusable code and a `__main__` guard to run a script's
entry point only when the file is executed directly.

Syntax

Diagram

railroad-ebnf-beta title "Imports and Script Entry Point" import_statement = "import" "module" ["as" "alias"] | "from" "module" "import" "member" ["as" "alias"] ; member_import = "from" "module" "import" "member" ["," "member"]* ; script_structure = import_statement* "constants" "function definitions" main_guard ; main_guard = "if" "__name__" "==" '"__main__"' ":" "main" "(" ")" ;

Examples

```
# Each form binds a name in this module's namespace; comments show how to use it.
import math
# Binds the name math; access members as math.sqrt(...), math.pi.
square_root = math.sqrt(25)

import math as maths
# Binds the alias maths; access members as maths.sqrt(...), maths.pi.
circle_area = maths.pi * 2 ** 2

from math import sqrt
# Binds sqrt directly; call sqrt(...) without a module prefix.
another_root = sqrt(36)

from math import pi as circle_pi
# Binds the alias circle_pi directly; use circle_pi without a module prefix.
circumference = 2 * circle_pi * 2
# square_root is 5.0; circle_area is approximately 12.57; another_root is 6.0.
# circumference is approximately 12.57.
```

```
# The main_guard prevents main() from running when this module is imported.
# Imports bind names in this module; from-imports do not bind math unless imported.
def main() -> None:
  """Run the host report."""
  print("Host report")  # Host report


if __name__ == "__main__":
  main()
```

```
# Typical package layout:
# Example import: from host_tools import checks; call checks.function_name().
project/
  main.py
  host_tools/
    __init__.py
    checks.py
```

Notes

- A module is a `.py` file.
- `import module` creates a name for the module in the importing module's
  namespace; access its functions and variables as `module.member`.
- `import module as alias` creates the alias; access members as `alias.member`.
- `from module import member` creates a direct name for that member, while
  `from module import member as alias` creates only the alias.
- An import inside a function creates a function-local name; a top-level import
  creates a module-level name.
- Avoid `from module import *` because it obscures where names came from.
- Keep imports near the top of a module and reusable functions in modules.

Variable Scope

Purpose

Global scope belongs to the module, local scope belongs to a function call, and nonlocal scope refers
to an enclosing function's variable.

Syntax

Examples

```
host_count = 2  # Global: module-level name.

def report_scope() -> None:
  host_count = 3  # Local: new name for this function call.
  print(host_count)  # 3


def add_host() -> None:
  global host_count
  host_count += 1  # Changes the module-level name.


def make_counter():
  count = 0  # Enclosing function scope.

  def next_count() -> int:
    nonlocal count
    count += 1  # Changes the enclosing function's name.
    return count

  return next_count


report_scope()
add_host()
counter = make_counter()
print(host_count)  # 3
print(counter(), counter())  # 1 2
```

Notes

- A module-level assignment creates a global name for that module.
- An assignment inside a function creates a local name unless `global` or `nonlocal` changes the binding rule.
- `global name` makes assignments in the current function update the module-level
  name.
- `nonlocal name` makes assignments in a nested function update the nearest
  enclosing function's name; it cannot refer to a module-level name.
- Python resolves reads through the nearest enclosing scope outward: local,
  enclosing function, global module, then built-ins.
- Prefer passing values and returning results over changing global state.

uv Projects and Virtual Environments

Purpose

Use a project manager and virtual environment to keep each project's Python
version and third-party packages isolated, recorded, and reproducible.

Syntax

Diagram

railroad-ebnf-beta title "Basic uv Project Workflow" project_setup = "uv init" "project name" ; dependency_change = "uv add" "package" | "uv add --dev" "development tool" | "uv remove" "package" ; environment_setup = "uv sync" ; run_command = "uv run" command ; standalone_environment = "uv venv" ["--python" "version"] ;

Examples

```
# Run from the project folder: init creates files, add records dependencies,
# sync prepares the environment, and run executes commands in that environment.
uv init host_report
cd host_report
uv add requests
uv add --dev pytest
uv sync
uv run python main.py
uv run pytest  # Runs the test suite and reports the test results.
```

```
# For a standalone environment without project metadata, use uv venv.
uv venv --python 3.14
# Creates a standalone .venv using Python 3.14.
```

Notes

- `pyproject.toml` stores project metadata and direct dependency declarations;
  `uv.lock` records the resolved dependency versions.
- The project environment is normally stored in `.venv`. `uv run` uses it without requiring manual activation.
- `uv add --dev pytest` marks pytest as a development dependency; a runtime
  package such as `requests` is added with `uv add requests`.
- After cloning or receiving a project, `uv sync` recreates its environment from
  the project files and lockfile.

pytest Basics

Purpose

Use pytest tests to check expected program behavior automatically and repeatably
as code changes.

Syntax

Diagram

railroad-ebnf-beta title "pytest Test Structure" test_file = "test_" "module name" ".py" ; test_function = "def" "test_" "behavior name" "(" ")" ":" arrange act assert ; arrange = "set up input values" ; act = "call function under test" ; assert = "assert" "actual result" "comparison" "expected result" ; test_run = "uv run pytest" ["test file or folder"] ;

Examples

```
# test_checks.py
# The test_ names follow pytest discovery; the body uses Arrange, Act, Assert.
from checks import is_port_available


def test_port_not_in_open_ports_is_available():
  open_ports = [22, 443]

  result = is_port_available(80, open_ports)

  assert result is True  # Passes because port 80 is not open.
```

Notes

- Test file names start with `test_`; test function names also start with `test_`.
- Use `assert` to compare actual behavior with expected behavior.

- Run project tests with `uv run pytest`; run one file with `uv run pytest tests/test_checks.py`.